

Lab 1: Introduction to C Programming

Computer Programming () |

Duration: 2 hours (in-lab) | **Topic:** IDE workflow, basic program structure, `printf/scanf`

Objectives

- Use an IDE (e.g. Dev-C++, Code::Blocks, VS Code) to create, compile, and run a C program.
- Read the structure of a minimal C program (`#include`, `main`, `return`).
- Perform basic arithmetic in expressions and assign results to variables.
- Use `printf` to display output and `scanf` to read input.

Quick Reference

Minimal program.

```
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

Common format specifiers.

%d	int	%f	float
%lf	double	%c	char
%s	string (char[])	%%	literal %

Input. Use `scanf("%d", &x);` – the `&` (address-of) is required for numeric inputs. For a single character after a previous `scanf`, use " %c" (with leading space) to skip the leftover newline.

Workflow. Edit → Save as `name.c` → Compile → Run. A compile error means your source did not produce an executable; read the first error message first.

Quick Experiments (Try It)

Try It 1: What happens with a wrong format specifier?

Type and run this program exactly as shown:

```
#include <stdio.h>
int main(void) {
    double x = 3.14;
    printf("x = %d\n", x);    /* %d expects int, not double */
    return 0;
}
```

1. Does the compiler give a warning, an error, or nothing?
2. What does the output look like? Is it what you expected?
3. Change `%d` to `%f`. Does the output change?
4. What rule does this teach about matching format specifiers to variable types?

Try It 2: Explore escape sequences

In your “Hello, world!” program replace `\n` with each of the following one at a time, predict the output first, then run:

- `\t` (horizontal tab)
- `\n\n` (two newlines)
- `\\` (literal backslash)
- `%%` (literal percent sign, inside the format string)

Record the one result that surprised you most.

Try It 3: Integer division – change one character, different result

Start with:

```
#include <stdio.h>
int main(void) {
    printf("%d\n", 7 / 2);
    return 0;
}
```

1. Run it. What is printed?
2. Change 2 to 2.0. What is printed now? Why?
3. What format specifier do you need to use to see the decimal places?

In-Lab Exercises (target: 2 hours)**In-Lab Exercise 1: Hello + Your Info (15 min)**

Write a program that prints three lines: (1) Hello, world!, (2) your full name, (3) your student ID. **Goal:** verify the toolchain (compile + run) works end-to-end.

In-Lab Exercise 2: Simple Calculator (35 min)

Read two integers from the user and print their sum, difference, product, integer quotient, and remainder.

- Label each line of output clearly, e.g. Sum: 12 + 5 = 17.
- What happens if the user enters 0 for the second number? Try it and note the behavior in a comment.

In-Lab Exercise 3: Unit Conversion: Celsius → Fahrenheit (30 min)

Read a temperature in Celsius (`double`) and print the equivalent in Fahrenheit using $F = \frac{9}{5}C + 32$. Print the result with two digits after the decimal point (`%.2f`). Test with $C = 0$, 100, and -40 .

In-Lab Exercise 4: ASCII Name Tag (40 min)

Ask for the user’s name (one word) and a single border character. Print a 3-line ASCII box that contains the name. Example:

```
#####
#  BMO  #
#####
```

Hint: use `scanf("%s", name)` for the name and `scanf(" %c", &ch)` for the character.

AI Collaboration

Try these prompts with an AI tool:

- “Explain what each line of a minimal C program does: `#include`, `main`, `return 0;`, and the semicolons.”
- “What is the difference between `%d`, `%f`, and `%lf` in `printf`? Give one example each.”
- “My program compiles but prints garbage when I use `%d` to print a `double`. Why?”

Critical check – verify, don’t just accept: Take the AI’s explanation of format specifiers and write a small program that intentionally mismatches `%d` with a `double`, `%f` with an `int`, and `%s` with a `char`. Run it and compare the actual output and compiler warnings against what the AI predicted. Note every discrepancy.

Know the limit: AI tools explain format specifiers confidently but sometimes give example code that compiles with a *warning* rather than an error – always read your compiler’s full output yourself, never assume the AI’s snippet is warning-free.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Output of a short program

What does the following program print? Write the exact output.

```
#include <stdio.h>
int main(void) {
    int a = 7, b = 2;
    printf("%d %d\n", a / b, a % b);
    printf("%d\n", a + b * 3);
    return 0;
}
```

Trace & Predict 2: Format specifier mismatch

Predict what is printed (and why it may look wrong):

```
#include <stdio.h>
int main(void) {
    double x = 3.5;
    printf("x = %d\n", x);
    return 0;
}
```

Find the Bug 1: Two errors

Find and fix *all* errors. Rewrite the corrected program on your paper.

```
#include <stdio.h>
```

```
int main(void) {  
    int n;  
    printf("Enter a number: ")  
    scanf("%d", n);  
    printf("You entered %d\n", n);  
    return 0;  
}
```

Write Code on Paper 1: Average of three integers

Write a complete C program (handwritten) that reads three integers from the user and prints their average as a floating-point number with two decimal places. Show every **#include**, the signature of **main**, and the **return** statement.

Write Code on Paper 2: Box perimeter and area

Write a complete C program that reads the width and height of a rectangle (as integers) and prints both its perimeter and its area on separate lines.

Submission. Hand in your paper-work at the start of the next lab. Save your in-lab source files as `lab1_ex1.c`, `lab1_ex2.c`, ... and submit them on the course site.